

Implementação de Máquinas Abstratas: AFD, Autômato de Pilha e Máquina de Turing

Cauã de Moraes Furtado, Anuska Cristal Quintela Campos de Oliveira,
Rhuan Cesar de Jesus Oliveira

Faculdade Cotemig — Sistemas de Informação / Ciência da Computação

Belo Horizonte, MG — Brasil

Fundamentos Teóricos da Computação

2026

Abstract

Este relatório documenta a especificação conceitual, as decisões de projeto e os resultados de implementação de três classes de máquinas abstratas da Hierarquia de Chomsky: Autômatos Finitos Determinísticos (AFD), Autômatos de Pilha não-determinísticos (AP) e Máquinas de Turing (MT). Todas as máquinas foram implementadas em *C#* sobre a plataforma .NET, priorizando o mapeamento direto entre as tuplas formais matemáticas e estruturas de dados nativas. São apresentadas as fundamentações teóricas, simulações passo a passo, questões reflexivas, detalhes de implementação e baterias de testes.

1 Introdução

No âmbito da Teoria da Computação e da Análise de Linguagens Formais, a Hierarquia de Chomsky estabelece uma classificação rigorosa dos modelos matemáticos computacionais de acordo com seu poder de expressão e restrição gramatical (MENEZES, 2010). O estudo prático dessas estruturas permite relacionar conceitos teóricos abstratos — como estados, transições e fitas infinitas — com a engenharia de software contemporânea, validando a premissa de que a escolha do modelo ideal depende diretamente da complexidade do problema a ser resolvido (HOPCROFT; MOTWANI; ULLMAN, 2003).

O objetivo deste relatório é documentar a especificação conceitual, as decisões de projeto e os resultados de implementação de três classes cruciais de máquinas abstratas:

- **Autômatos Finitos Determinísticos (AFD):** reconhecedores de Linguagens Regulares (Tipo 3), que operam com memória interna finita sob a forma de estados.
- **Autômatos de Pilha Não-Determinísticos (AP):** reconhecedores de Linguagens Livres de Contexto (Tipo 2), configurados com critério de aceitação por pilha vazia para expandir a capacidade de armazenamento local.
- **Máquinas de Turing (MT):** reconhecedores e transdutores de Linguagens Recursivamente Enumeráveis (Tipo 0), representando o limite superior da computação clássica.

A arquitetura de software foi inteiramente desenvolvida utilizando a linguagem C# e a plataforma .NET, priorizando o mapeamento direto entre as tuplas formais matemáticas e as estruturas de dados nativas da linguagem (como dicionários hash, conjuntos e pilhas), garantindo legibilidade, modularidade e forte aderência teórica.

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um Autômato Finito Determinístico é definido como uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde:

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de entrada;
- $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados de aceitação.

Utilizando como exemplo de testes manuais e de validação teórica, a linguagem delimitada para implementação foi:

$$L_1 = \{ w \in \{a, b\}^* \mid w \text{ termina com } ab \}$$

q_0 (**Estado Inicial**): indica que o padrão ainda não começou a ser formado ou foi interrompido. Ao ler o símbolo **a**, o autômato identifica o possível início do sufixo alvo e transiciona para q_1 . Ao ler o símbolo **b**, não há avanço no reconhecimento do padrão, obrigando a máquina a permanecer no próprio q_0 .

q_1 : armazena a informação de que o último caractere lido foi um **a**. Se a fita apresentar um novo **a**, a máquina permanece em q_1 , pois este novo caractere substitui o anterior como

candidato ao início de “ab”. Contudo, se a fita apresentar um **b**, o padrão é completado e a máquina avança para o estado q_2 .

q_2 (**Estado de Aceitação**): ao atingir este estado, a condição da linguagem foi momentaneamente satisfeita. Como o autômato deve consumir toda a cadeia, se houver mais símbolos, a leitura continua. A leitura de um novo **a** quebra o sufixo atual, mas inicia um novo (retornando a q_1). A leitura de um **b** destrói o padrão completamente, obrigando a máquina a regressar à estaca zero (q_0).

Para comprovar o determinismo e a precisão do modelo abstrato, foi simulado o processamento passo a passo da fita contendo a cadeia $w = aabab$:

1. Início: $(q_0, aabab)$
2. Passo 1: $\delta(q_0, a) = q_1. (q_1, abab)$
3. Passo 2: $\delta(q_1, a) = q_1. (q_1, bab)$
4. Passo 3: $\delta(q_1, b) = q_2. (q_2, ab)$
5. Passo 4: $\delta(q_2, a) = q_1. (q_1, b)$
6. Passo 5: $\delta(q_1, b) = q_2. (q_2, \varepsilon)$

Após consumir o último caractere, alcançando a cadeia vazia, a máquina permanece no estado q_2 . Dado que $q_2 \in F$, a linguagem reconhece e aceita a palavra.

2.1.1 Questões Reflexivas — Parte 1

1. Por que a linguagem L_1 é regular? Apresente a expressão regular equivalente. A linguagem L_1 é classificada como regular porque pode ser perfeitamente expressa e reconhecida por um modelo de memória estritamente finita, sem a necessidade de mecanismos auxiliares de contagem ou de empilhamento de dados. Conforme postulado pelo Teorema de Kleene (MENEZES, 2010), toda linguagem que pode ser denotada por uma expressão regular pode ser reconhecida por um AFD. No caso de L_1 , a máquina necessita de uma quantidade constante e invariável de memória; ela só precisa “lembrar-se” das últimas duas posições da cadeia analisada para validar o sufixo, não importando se a cadeia possui três ou três milhões de caracteres. Qualquer combinação arbitrária de caracteres prévios é absorvida pelo modelo sem gerar sobrecarga estrutural. A expressão regular equivalente que descreve matematicamente esse comportamento é $(a \cup b)^*ab$, o que evidencia que a palavra pode conter qualquer arranjo de ocorrências de **a** e **b** no prefixo, contanto que seja compulsoriamente finalizada pela concatenação exata da substring “ab” (SIPSER, 2013).

2. Descreva formalmente a 5-tupla do AFD construído, justificando a escolha de cada estado. A construção do AFD para a linguagem L_1 corporifica-se na seguinte 5-tupla:

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\}).$$

A delimitação do alfabeto como $\Sigma = \{a, b\}$ obedece à especificação do problema. A modelagem lógica resultou em apenas três estados, um número mínimo e suficiente garantido pelo Lema de Myhill-Nerode para distinguir as classes de equivalência desta linguagem específica. O estado q_0 atua como um sumidouro temporário e ponto de partida; sua escolha justifica-se pela necessidade de um estágio onde nenhum progresso rumo à subcadeia “ab” tenha sido alcançado. A transição para q_1 justifica-se como o armazenamento lógico temporal do evento “o caractere a foi lido”, marcando um estado de expectativa. Por fim, q_2 é justificado como a concretização do reconhecimento; ele é o único estado promovido ao conjunto $F = \{q_2\}$, pois sua ativação atesta formalmente e unicamente que a transição lógica do estado de expectativa para a leitura sequencial da letra b ocorreu, respeitando rigorosamente o sufixo terminal exigido pela linguagem (LEWIS; PAPADIMITRIOU, 1997).

3. O que aconteceria se a função δ não fosse total? Como você tratou entradas inválidas (símbolos fora do alfabeto)? A função de transição δ em um AFD possui, por definição formal, o dever de mapear o produto cartesiano $Q \times \Sigma$ de forma exaustiva. Se essa função não fosse total — ou seja, se existisse algum par composto por um estado e um símbolo para o qual não houvesse uma transição explicitamente definida —, o autômato entraria em uma condição de indefinição lógica, resultando em uma “parada anormal”. Computacionalmente, isso violaria o determinismo estrito da máquina, pois o processamento travaria antes de exaurir toda a fita de entrada, falhando em emitir um veredito formal e conclusivo de aceitação ou rejeição (HOPCROFT; MOTWANI; ULLMAN, 2003).

Na engenharia do software desenvolvido, essa vulnerabilidade teórica foi tratada em duas frentes estruturais complementares. Primeiramente, para lidar com entradas inválidas contendo símbolos que não pertencem ao alfabeto, implementou-se uma barreira de validação preventiva no início do laço principal de varredura. O algoritmo inspeciona cada caractere de forma isolada antes de tentar processá-lo; ao detectar um elemento anormal à linguagem, a execução é sumariamente abortada, o sistema emite um alerta de rejeição e a integridade da tupla matemática é preservada contra a injeção de dados espúrios.

Em segundo lugar, para contornar a possível omissão de transições lícitas na configuração do autômato (o que caracterizaria uma função não-total operando sobre símbolos válidos), a arquitetura do código utiliza um mecanismo de busca segura na estrutura de

dicionário que armazena as regras de transição. Em vez de acessar a memória de forma cega e provocar uma falha abrupta no programa, o sistema interroga o dicionário sobre a existência de um destino para o estado e o símbolo atuais. Se o mapeamento não for encontrado, o software assume nativamente um desvio lógico para um “estado morto” virtual. Essa abordagem encerra o rastreamento de forma elegante e decreta a rejeição da cadeia com segurança, eliminando a necessidade de instanciar e alocar na memória dezenas de transições para um estado de erro físico.

2.2 Autômato de Pilha com Reconhecimento por Pilha Vazia

Um autômato de pilha é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, onde:

- Q é um conjunto finito de estados;
- Σ é o alfabeto de entrada;
- Γ é o alfabeto da pilha;
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$ é a função de transição;
- $q_0 \in Q$ é o estado inicial;
- $Z_0 \in \Gamma$ é o símbolo inicial da pilha;
- o conjunto de estados de aceitação é $\emptyset$ (aceitação por pilha vazia).

Utilizando como escopo de validação prática e testes, o modelo foi configurado e homologado para o reconhecimento de uma linguagem livre de contexto determinística:

$$L_2 = \{ a^n b^n \mid n \geq 1 \}$$

A estratégia utiliza dois estados operacionais (q_0 e q_1). No estado inicial q_0 , para cada caractere **a** lido na fita, o autômato armazena a ocorrência empilhando o símbolo **a** sobre a memória, preservando o marcador de fundo Z_0 . No momento em que o primeiro caractere **b** é interceptado, a máquina realiza uma transição para o estado q_1 e desempilha o primeiro **a** do topo. O autômato permanece retido em q_1 consumindo os caracteres **b** subsequentes e eliminando um **a** correspondente para cada leitura. Ao término da cadeia de entrada, uma transição vazia (ε) é acionada: se o topo expuser o marcador inicial Z_0 , este é removido, deixando a pilha vazia e consumando a aceitação.

Para demonstrar a exatidão do modelo abstrato, foi simulada a entrada da cadeia $w = aaabbb$, pertencente a L_2 . As descrições rastreadas foram representadas pela tríade (Estado Atual, Cadeia Restante, Conteúdo da Pilha — onde o caractere mais à esquerda representa o topo):

1. Início: $(q_0, aaabbb, Z_0)$
2. Passo 1: lê o primeiro **a** → símbolo é empilhado: $(q_0, aabbb, aZ_0)$
3. Passo 2: lê o segundo **a** → símbolo é empilhado: $(q_0, abbb, aaZ_0)$
4. Passo 3: lê o terceiro **a** → símbolo é empilhado: $(q_0, bbb, aaaZ_0)$
5. Passo 4: lê o primeiro **b** → símbolo é desempilhado e muda para o estado q_1 : (q_1, bb, aaZ_0)
6. Passo 5: lê o segundo **b** → símbolo é desempilhado: (q_1, b, aZ_0)
7. Passo 6: lê o terceiro **b** → símbolo é desempilhado: (q_1, ε, Z_0)
8. Passo 7: como a fita de entrada é ε , o autômato processa a regra de fechamento por vazio. Z_0 é desempilhado: $(q_1, \varepsilon, \varepsilon)$

Como a cadeia inteira é processada e a fita auxiliar é reduzida para vazio (ε), a palavra w é aceita por pilha vazia.

2.2.1 Questões Reflexivas — Parte 2

1. Por que L_2 não pode ser reconhecida por um AFD? Demonstre usando o Lema do Bombeamento para linguagens regulares. A linguagem $L_2 = \{a^n b^n \mid n \geq 1\}$ não pode ser reconhecida por um Autômato Finito Determinístico porque exige uma capacidade de memória infinita e dinâmica para contabilizar e emparelhar a quantidade de caracteres do primeiro bloco com os do segundo bloco. Por definição, um AFD possui um conjunto fixo e limitado de estados internos, o que o impede de “contar” uma quantidade arbitrária e ilimitada de símbolos. Para demonstrar formalmente essa limitação, aplica-se o Lema do Bombeamento para linguagens regulares.

Suponha que L_2 seja uma linguagem regular. Portanto, deve existir uma constante de bombeamento p tal que qualquer cadeia $s \in L_2$ com comprimento $|s| \geq p$ possa ser dividida em três partes, $s = xyz$, atendendo às condições de que $|xy| \leq p$, $|y| > 0$ e, para todo $i \geq 0$, a cadeia bombeada $xy^i z \in L_2$.

Escolhe-se a cadeia específica $s = a^p b^p$, cuja pertinência a L_2 é imediata e cujo comprimento $|s| = 2p$ é claramente maior ou igual a p . Dado que a restrição do lema impõe que a subcadeia inicial $|xy|$ não deve exceder o valor de p , a parte identificada como y (que representa o segmento a ser bombeado) deve ser constituída obrigatoriamente e exclusivamente por caracteres **a**. Sendo assim, podemos definir $y = a^k$, onde $k \geq 1$ devido à condição de que y não pode ser vazio. Ao realizarmos o bombeamento para o valor de $i = 2$, gerando a nova cadeia $xy^2 z$, o número de caracteres **a** é incrementado pelo fator k , transformando a palavra resultante em $a^{p+k} b^p$. Como o valor de k é estritamente maior que zero, o número de símbolos **a** passa a ser estritamente maior que o número

de símbolos $\mathbf{b}$ ($p + k \neq p$). Essa assimetria quantitativa viola a regra de formação da linguagem original, o que significa que a cadeia bombeada $xy^2z \notin L_2$. Essa contradição quebra a suposição inicial, provando matematicamente que L_2 não é uma linguagem regular e, conseqüentemente, não pode ser reconhecida por nenhum AFD (HOPCROFT; MOTWANI; ULLMAN, 2003).

2. A aceitação por pilha vazia e por estado final são equivalentes em poder de reconhecimento? Demonstre conceitualmente. Sim, os critérios de aceitação por pilha vazia e por estado final são perfeitamente equivalentes em termos de poder de reconhecimento de linguagens. Ambos os modelos definem e delimitam com exatidão a mesma classe de linguagens formais, que são as Linguagens Livres de Contexto (Tipo 2 na Hierarquia de Chomsky). A demonstração conceitual dessa equivalência fundamenta-se na possibilidade de realizarmos uma conversão estrutural bidirecional entre os dois tipos de autômatos, provando que qualquer máquina construída sob um critério pode ser mapeada para o outro sem perda de generalidade (MENEZES, 2010).

Para converter um autômato que aceita por estado final em um que aceita por pilha vazia, adiciona-se um novo estado inicial e um novo marcador de fundo de pilha. Esse marcador serve para blindar a máquina contra um esvaziamento precoce acidental durante o processamento regular. Além disso, criam-se transições vazias (ε) a partir de todos os antigos estados de aceitação originais, direcionando o fluxo para um novo estado especializado cuja única função é esvaziar sistematicamente todos os símbolos remanescentes da pilha. Inversamente, para transformar um autômato que aceita por pilha vazia em um modelo de aceitação por estado final, insere-se de igual modo um novo símbolo de fundo e um novo estado inicial. A função de transição é então configurada para interceptar o momento exato em que esse marcador de proteção torna-se o único elemento sobrevivente na memória, indicando que o autômato original teria limpado sua pilha por completo. Ao detectar essa configuração, a máquina dispara uma transição para um novo estado terminal, que é explicitamente declarado como o estado de aceitação do sistema. Portanto, a equivalência é absoluta, alterando-se apenas o mecanismo de sinalização do fim da computação (LEWIS; PAPADIMITRIOU, 1997).

3. Qual o papel do símbolo inicial de pilha na implementação? Na engenharia do simulador desenvolvido, o símbolo inicial da pilha (identificado no projeto pelo caractere Z) desempenha um papel de extrema relevância como balizador de consistência lógica e controle de fluxo do não-determinismo. Matematicamente, a pilha de um autômato abstrato inicia sua operação contendo esse marcador de fundo, impedindo que a máquina execute leituras ou operações de desempilhamento em uma estrutura vazia no início do programa. No código implementado, o caractere Z atua primordialmente como uma âncora de segurança que impede a ocorrência de erros de subfluxo de dados (*stack underflow*)

durante a simulação das ramificações de busca.

2.3 Máquina de Turing

Uma Máquina de Turing é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, onde:

- Q é o conjunto finito de estados;
- Σ é o alfabeto de entrada ($\sqcup \notin \Sigma$);
- $\Gamma \supseteq \Sigma$ é o alfabeto da fita ($\sqcup \in \Gamma$);
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ é a função de transição;
- q_0 é o estado inicial;
- q_{accept} é o estado de aceitação;
- q_{reject} é o estado de rejeição ($q_{reject} \neq q_{accept}$).

Utilizando como base conceitual a implementação do projeto, a máquina foi modelada para reconhecer a linguagem sensível ao contexto L_4 :

$$L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$$

- **Busca e Marcação** (q_0, q_1, q_2): no estado inicial q_0 , a máquina localiza o primeiro a não marcado, sobrescreve-o com o símbolo X e move-se à direita para q_1 . Em q_1 , o cabeçote ignora outros a's e eventuais marcas Y até encontrar o primeiro b disponível, sobrescrevendo-o com Y e passando a q_2 . Em q_2 , o processo se repete: ignoram-se os b's e Z's até localizar o primeiro c, que é reescrito como Z. Após marcar a trinca, a máquina entra no estado de retrocesso q_3 .
- **Retrocesso** (q_3): o cabeçote inverte seu sentido de leitura e move-se continuamente para a esquerda (L), saltando todos os caracteres do alfabeto e as marcas (a, b, X, Y, Z) até interceptar o símbolo em branco ($\sqcup$) que antecede a palavra. Ao encontrá-lo, move-se um passo à direita e retorna a q_0 para iniciar um novo ciclo.
- **Validação Final** (q_4): se, ao iniciar um novo ciclo em q_0 , a máquina ler imediatamente a marca Y (indicando que não há mais a's a serem marcados), ela assume que a palavra pode ter acabado e transiciona para o estado de homologação q_4 . Neste estado, o cabeçote varre a fita à direita verificando se restaram apenas marcas Y e Z. Se atingir o final da fita (símbolo em branco) sem encontrar b's ou c's sobressalentes, transiciona para q_{accept} .

Para demonstrar a mecânica de fita, simulamos a computação da cadeia mínima $w = abc$. A notação utiliza colchetes $[]$ para indicar a posição atual do cabeçote sobre a fita infinita (onde os espaços não exibidos são brancos $\sqcup$):

1. Início (q_0): fita contém $\sqcup[a]bc\sqcup$. A máquina lê **a**, escreve **X**, move R e vai para q_1 .
2. Passo 1 (q_1): a máquina avança lendo **b**, escreve **Y**, move R e vai para q_2 . Fita: $\sqcup X[c]\sqcup$.
3. Passo 2 (q_2): a fita contém $\sqcup XY[c]\sqcup$. A máquina lê **c**, escreve **Z**, move L e vai para q_3 .
4. Passo 3 (q_3): a fita contém $\sqcup X[Y]Z$. A máquina lê **Y**, escreve **Y**, move L e continua em q_3 .
5. Passo 4 (q_3): a fita contém $\sqcup[X]YZ$. A máquina lê **X**, escreve **X**, move L e continua em q_3 .
6. Passo 5 (q_3): a fita contém $[\sqcup]XYZ$. A máquina lê o branco $\sqcup$, escreve $\sqcup$, move R e retorna a q_0 .
7. Passo 6 (q_0): a fita contém $\sqcup[X]YZ$ — o cabeçote avançou sobre o **X**. Lê **X**, move R e mantém q_0 . A fita fica $\sqcup X[Y]Z$.
8. Passo 7 (q_0): lê **Y**. Como os **a**'s acabaram, move R e vai para q_4 . Fita: $\sqcup XY[Z]$.
9. Passo 8 (q_4): lê **Z**, move R e mantém q_4 . Fita: $\sqcup XYZ[\sqcup]$.
10. Passo 9 (q_4): lê o branco $\sqcup$, move R e transiciona para q_{accept} .

A máquina atingiu o estado de aceitação após marcar todos os elementos e homologar a fita, aceitando corretamente a palavra.

2.3.1 Questões Reflexivas — Parte 3

1. Por que L_4 não pode ser reconhecida por um Autômato de Pilha? Qual propriedade da MT possibilita o seu reconhecimento? A linguagem $a^n b^n c^n$ não é livre de contexto, o que se demonstra pelo Lema do Bombeamento para linguagens livres de contexto. Um Autômato de Pilha dispõe de uma única pilha, cuja política LIFO permite correlacionar apenas duas grandezas: ele empilha os **a** e desempilha contra os **b**, mas ao esvaziar a pilha perde toda a informação de contagem e não consegue mais compará-la com os **c**.

O que a Máquina de Turing acrescenta é uma fita de leitura e escrita com movimento bidirecional do cabeçote. Essa capacidade de reescrever símbolos — marcar e visitar — e de fazer múltiplas varreduras confere à MT uma memória de trabalho irrestrita. É exatamente essa propriedade que viabiliza a verificação simultânea das três

contagens. Em termos da Hierarquia de Chomsky, L_4 é sensível ao contexto (Tipo 1), reconhecível por Máquina de Turing, porém fora da classe livre de contexto (Tipo 2).

2. Quantos estados foram necessários para a MT de L_4 ? Descreva a estratégia de “marcação” adotada. A MT de L_4 usa 7 estados no total: cinco estados de trabalho (q_0, q_1, q_2, q_3, q_4) mais os dois estados de parada explícitos q_{accept} e q_{reject} . Se contarmos apenas os estados que efetivamente processam a fita, são 5.

A estratégia é de marcação por correspondência um-para-um-para-um, e funciona em rodadas. A ideia central é que o alfabeto da fita Γ é maior que o de entrada Σ : além de $\{a, b, c\}$, ele inclui os símbolos auxiliares $\{X, Y, Z\}$, que servem para “riscar” um símbolo já contabilizado sem apagá-lo. Cada rodada consome exatamente um a, um b e um c:

- q_0 — varre da esquerda procurando o primeiro a não marcado. Ao achá-lo, reescreve $a \rightarrow X$ e passa a q_1 . (Se em vez de a encontrar um Y, significa que os a acabaram, e ele desvia para q_4 para a verificação final.)
- q_1 — avança à direita, ignorando os a restantes e os Y já marcados, até o primeiro b não marcado. Reescreve $b \rightarrow Y$ e passa a q_2 .
- q_2 — continua à direita, ignorando b e Z, até o primeiro c não marcado. Reescreve $c \rightarrow Z$ e passa a q_3 .
- q_3 — rebobina o cabeçote para a esquerda, atravessando tudo, até bater no branco inicial; então retorna a q_0 para iniciar a próxima rodada.

Esse ciclo $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_0$ se repete enquanto houver a não marcados. Quando todos os a viraram X, o q_0 cai em q_4 , que faz uma varredura de verificação: ele só percorre Y e Z e exige terminar em branco. Se durante essa varredura sobrar qualquer b ou c não marcado (ou um a), não existe transição definida e a máquina cai em q_{reject} . Caso contrário, chega a q_{accept} .

O ponto formal interessante é por que isso garante $n_a = n_b = n_c$: como cada rodada marca obrigatoriamente um de cada (e falha se faltar algum), a aceitação só ocorre se as três quantidades se esgotam exatamente juntas. É a marcação que substitui a “contagem” que um Autômato de Pilha não consegue manter para três grandezas simultâneas.

3. Um computador moderno é mais poderoso que uma Máquina de Turing? Justifique à luz da Tese de Church-Turing. Não, sob o ponto de vista estrito da Teoria da Computabilidade, um computador moderno não é formalmente mais poderoso do que uma Máquina de Turing. A Tese de Church-Turing, pilar central da Ciência da Computação teórica, postula que qualquer processo algorítmico, função matemática ou cálculo intuitivo que possa ser concebido e resolvido mecanicamente pode ser computado

por uma Máquina de Turing padrão (SIPSER, 2013). Os computadores contemporâneos, baseados na arquitetura de von Neumann (utilizando linguagens de altíssimo nível, memória RAM, registradores quânticos ou milhares de núcleos de processamento paralelo), alteram drasticamente a eficiência e a complexidade de tempo das operações, mas não expandem a classe de problemas que podem ser matematicamente solucionáveis. Se um problema é classificado como incomputável ou indecidível para uma MT (como o Problema da Parada), ele continuará sendo insolúvel para o supercomputador mais avançado da Terra. Na verdade, por ser uma abstração matemática munida de uma fita rigorosamente infinita, a Máquina de Turing possui, na teoria, mais capacidade de armazenamento do que qualquer hardware físico existente, que é sempre limitado por barreiras termodinâmicas e quânticas (GUEDES; LULA JÚNIOR, 2009). A superioridade dos sistemas modernos reside, portanto, na performance empírica e na usabilidade, sendo computacionalmente equivalentes e submissos aos mesmos limites teóricos estabelecidos por Alan Turing em 1936.

3 Descrição da Implementação

A materialização dos modelos matemáticos em software foi desenvolvida na plataforma .NET com a linguagem C#, utilizando os princípios da Programação Orientada a Objetos. O projeto isolou a lógica de cada máquina abstrata, garantindo que as estruturas de dados refletissem com precisão as definições formais (tuplas) e suportassem a validação das linguagens estipuladas.

3.1 Implementação do Autômato Finito Determinístico (AFD)

- **Conjuntos Matemáticos:** os estados operacionais (Q) e os alfabetos (Σ) foram implementados utilizando a estrutura `HashSet`. Esta escolha garante a unicidade dos elementos (impedindo duplicações, conforme a teoria dos conjuntos) e proporciona um desempenho de complexidade constante $O(1)$ nas validações de leitura.
- **Função de Transição (δ):** mapeada através de um `Dictionary<(string, char), string>`. A associação direta de uma chave composta (estado atual e símbolo lido) a um único estado de destino reflete o determinismo estrito do modelo.
- **Robustez e Estado Morto:** para garantir que a função seja simulada de forma segura (lidando com a falta de mapeamento total), o algoritmo intercepta caracteres inválidos ou transições inexistentes e redireciona o fluxo para um “estado morto” virtual. Isto rejeita a cadeia com segurança, evitando falhas abruptas no sistema.

3.2 Implementação do Autômato de Pilha (AP)

- **Memória LIFO e Transições:** a fita auxiliar foi incorporada pela classe `Stack<char>`. Para acomodar a complexidade da transição δ , utilizou-se um dicionário avançado: `Dictionary<(string, char, char), List<(string, string)>`. Esta estrutura permite que uma mesma configuração de estado, entrada e topo da pilha aponte para uma lista de múltiplos destinos possíveis, o que é a essência do não-determinismo.
- **Simulação Não-Determinística:** como o processador físico é determinístico, o não-determinismo foi simulado algoritmicamente através de uma busca em profundidade com retrocesso (*backtracking*).
- **Cópia Defensiva da Pilha:** o maior desafio lógico foi garantir que um caminho de teste falhado não corrompesse a memória de outro. Para isso, implementou-se uma clonagem defensiva: a cada ramificação da recursão, a pilha atual é convertida em vetor, invertida para manter a integridade do topo e instanciada numa nova `Stack`. Isto garante isolamento total entre os caminhos computacionais explorados.

3.3 Implementação da Máquina de Turing (MT)

- **Virtualização da Fita Infinita:** arranjos lineares tradicionais não conseguem representar uma fita infinita bidirecional devido a restrições de memória e indexação (que não suportam índices negativos). A solução projetada foi utilizar um `Dictionary<int, char>` para a fita. O cabeçote navega por chaves inteiras positivas e negativas livremente; se consultar um índice inexistente, o sistema retorna de forma nativa o símbolo em branco ($\sqcup$), mimetizando perfeitamente o modelo de Alan Turing.
- **Determinismo e Movimento:** a função de transição associou a chave de leitura (estado, símbolo) a uma tupla tripla contendo as diretivas mecânicas da máquina: (novo estado, símbolo a reescrever, direção L/R).
- **Mitigação do Problema da Parada:** como as Máquinas de Turing estão sujeitas ao Problema da Parada (*Halting Problem*) e podem entrar em ciclos infinitos perante cadeias rejeitadas, a arquitetura incluiu um limite dinâmico de segurança (`_maxSteps`). Se o cabeçote exceder os passos permitidos sem atingir o estado de aceitação ou rejeição formal, o motor dispara uma exceção e aborta a execução, protegendo o hardware do computador hospedeiro.

4 Resultados e Testes

Esta seção consolida os resultados obtidos durante a bateria de testes operacionais executada sobre os simuladores. A metodologia consistiu em submeter as máquinas abstratas a um lote controlado de cadeias (válidas e inválidas) para comprovar empiricamente a aderência do software aos modelos matemáticos. Para as cadeias vazias, adotou-se a notação formal ε .

4.1 Testes do Autômato Finito Determinístico (AFD)

$$L_1 = \{ w \in \{a, b\}^* \mid w \text{ termina com } ab \}$$

Cadeia (w)	Tam.	Resultado	Parada	Justificativa
ab	2	ACEITO	q_2	Cadeia mínima atendida. Atinge o estado de aceitação na leitura exata do sufixo.
aab	3	ACEITO	q_2	Transição circular em q_1 pelo primeiro a , seguida do fechamento correto do sufixo.
bab	3	ACEITO	q_2	O prefixo b mantém a máquina em q_0 , reconhecendo o sufixo “ ab ” ao final.
$ababab$	6	ACEITO	q_2	Múltiplas ocorrências lidas; a leitura final encaixa perfeitamente na restrição em q_2 .
ba	2	REJEITADO	q_1	A palavra encerra-se no estado de expectativa ($q_1 \notin F$), faltando o b final.
ε	0	REJEITADO	q_0	A máquina não realiza transições, estacionando no estado inicial neutro.

4.2 Testes do Autômato de Pilha (AP)

$$L_2 = \{ a^n b^n \mid n \geq 1 \} \quad L_3 = \{ w \in \{a, b\}^* \mid w = w^R, |w| \geq 1 \}$$

Ling.	Cadeia	Tam.	Resultado	Parada	Justificativa
L_2	$aabb$	4	ACEITO	q_1	Paridade exata ($n = 2$). A pilha acumula e esvazia perfeitamente, consumindo o marcador Z_0 .
L_2	ab	2	ACEITO	q_1	Cenário de paridade mínima ($n = 1$). Execução linear concluída com pilha vazia.
L_2	ba	2	REJEITADO	q_0	Incompatibilidade estrutural. O b inicial não possui regra de desempilhamento para o topo Z_0 .
L_3	aba	3	ACEITO	q_1	Simetria ímpar identificada. A ramificação usou b como pivô, validando o espelhamento do a .
L_3	aab	3	REJEITADO	q_1	Assimetria estrutural detectada na segunda metade, impossibilitando o esvaziamento da pilha.

4.3 Testes da Máquina de Turing (MT)

$$L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$$

Cadeia (w)	Tam.	Resultado	Parada	Justificativa
abc	3	ACEITO	q_{accept}	Paridade validada ($n = 1$). O ciclo de varredura marcou os elementos e concluiu a homologação.
$aaabbbccc$	9	ACEITO	q_{accept}	Paridade validada ($n = 3$). A fita suportou os três ciclos completos de varredura e marcação.
$aaabbcc$	7	REJEITADO	Travamento	Falta de um c . A máquina tentou fechar a terceira trinca, mas encontrou um espaço em branco.
ab	2	REJEITADO	Travamento	Estrutura incompleta. Falha por ausência obrigatória do bloco de caracteres c .
ε	0	REJEITADO	Travamento	A restrição de quantidade mínima ($n \geq 1$) da linguagem não foi atendida.
ba	2	REJEITADO	Travamento	Violação de ordem. O primeiro caractere não é o início esperado do bloco a .
$abbc$	4	REJEITADO	q_{reject}	Excesso de caracteres originais detectados na fita durante a fase de validação final.

Cadeia (w)	Tam.	Resultado	Parada	Justificativa
$aaabbccc$	8	REJEITADO	Travamento	Faltou um b central. A varredura abortou ao não encontrar a correspondência para a trinca.

5 Análise Comparativa

A capacidade de cada máquina abstrata é definida pela sua arquitetura de memória, determinando o seu nível na Hierarquia de Chomsky.

O Autômato Finito Determinístico possui o menor poder computacional por não ter memória auxiliar. A sua “memória” restringe-se ao estado atual. Por isso, reconhece padrões fixos como L_1 (terminados em “ab”), mas é incapaz de processar linguagens que exijam contagem. O AFD falha em L_2 , pois não consegue registrar a quantidade do primeiro bloco de caracteres para compará-la com o segundo.

O Autômato de Pilha supera esta limitação ao introduzir uma memória infinita estruturada em formato LIFO (*Last-In, First-Out*). Com a pilha, o modelo consegue “contar” e reconhecer linguagens com simetria, validando L_2 (empilhando os **a**’s e desempilhando-os com os **b**’s) e L_3 (palíndromos). Contudo, como o acesso é restrito apenas ao topo da pilha, o AP é incapaz de lidar com múltiplas dependências simultâneas. Falha perante linguagens sensíveis ao contexto como L_4 : ao desempilhar os elementos no bloco **b**, a contagem inicial é destruída, inviabilizando a validação do bloco **c**.

A Máquina de Turing representa o topo da computação clássica. Rompe as limitações do AP substituindo a pilha por uma fita bidirecional de acesso aleatório. A cabeça de leitura pode avançar, retroceder e reescrever dados sem os destruir. Esta liberdade espacial permite à MT não só reconhecer as linguagens inferiores (L_1 , L_2 , L_3), mas também solucionar correlações múltiplas como L_4 , processando qualquer problema que possua uma solução algorítmica.

Em suma, o poder cresce na relação

$$P(\text{AFD}) \subset P(\text{AP}) \subset P(\text{MT}),$$

sendo esta evolução ditada exclusivamente pela liberdade e capacidade de manipulação da memória de cada modelo.

6 Conclusão

A realização deste trabalho final consolida e coroa todo o aprendizado teórico que construímos ao longo do semestre em sala de aula. Mais do que apenas analisar as tuplas da Hierarquia de Chomsky no quadro ou nos livros, codificar o comportamento dessas máquinas do zero foi o verdadeiro divisor de águas para a nossa compreensão.

A implementação em C# teve um papel fundamental nesse processo. Ao transformar definições matemáticas abstratas em estruturas de dados reais — mapeando conjuntos para `HashSets`, funções de transição para `Dictionary`s e memórias LIFO para `Stacks` —, a teoria ganhou forma lógica e visual. Lidar com o não-determinismo do Autômato de Pilha na prática, construindo algoritmos de *backtracking* e isolando a memória em cada recursão, deixou muito claro o alto custo computacional desse modelo. Da mesma forma, virtualizar a fita infinita da Máquina de Turing mostrou como a arquitetura de software pode contornar limitações físicas.

Por fim, o maior impacto deste trabalho é a mudança de perspectiva sobre a nossa própria área. É absolutamente genial — e até fascinante — perceber que toda a engenharia de software contemporânea continua estritamente submissa a modelos concebidos entre as décadas de 1930 e 1950. Independentemente de quão avançados sejam os nossos computadores modernos, a linguagem C# ou a plataforma .NET, os limites lógicos do que podemos ou não programar ainda são regidos pelas descobertas de Alan Turing e Noam Chomsky feitas há quase um século, numa época em que o computador pessoal sequer existia.

Compreender que a base matemática da computação permanece inabalável não apenas fortalece a nossa base técnica, mas nos torna estudantes e futuros cientistas da computação muito mais maduros, capazes de enxergar a raiz lógica por trás de qualquer tecnologia moderna.

Referências

GUEDES, Elloá B.; LULA JÚNIOR, Bernardo. *Autômatos finitos: com uma introdução aos autômatos finitos quânticos*. Campina Grande: EDUEFG, 2009.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à teoria de autômatos, linguagens e computação*. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elements of the theory of computation*. 2. ed. Upper Saddle River: Prentice Hall, 1997.

MENEZES, Paulo Blauth. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2010.

SIPSER, Michael. *Introduction to the theory of computation*. 3. ed. Boston: Cengage Learning, 2013.